CVIS

Connected Vehicle Intelligence System

Centralized Fleet Telematics, Cloud AI Reasoning & Cryptographic Wire Security

Comprehensive Technical Specification, Industrial Feasibility & CCNS Syllabus Mapping

Course:	Computer Communication & Network Security (CCNS)
Project Classification:	Academic Project — Industry-Inspired Electric Vehicle Architecture
Software Edition:	Version 3.0 (Autonomous Flight Recorder & Multi-Tier SaaS Architecture)
Technology Stack:	ESP32 Embedded · FastAPI (Python 3.12) · SQLite (WAL) · Ollama (CUDA) · Next.js 14
Transport Protocols:	Dual Runtime-Switchable HTTP REST $\Longleftrightarrow$ MQTT (QoS 0/1)
Security Controls:	Cryptographic HMAC-SHA256 · AEAD AES-256-GCM · Constant-Time Auth
Hardware Testbed:	NVIDIA GeForce RTX 4060 GPU (CUDA Accelerated Inference)

Architectural Dataflow Pipeline

ESP32 Thin Client / TCU

$\xrightarrow{\text{HTTP REST / MQTT (Runtime Switched)}}$ **FastAPI Ingest Gateway**

`HMAC-SHA256 Wire Integrity` | `AES-256-GCM Authenticated Encryption`

FastAPI Ingest $\xrightarrow{\text{High-Speed WebSocket Fan-out}}$ **Next.js Unified Frontend**

Role-Gated Portals: `/driver` (Telemetry) `/noc` (Network Controls) `/admin` (Security Audit)

Centralized Ollama AI ⟵ 17-Factor Physics & Anomaly Reasoning (GPU Accelerated)

Autonomous Flight Recorder Validated — Continuous Telemetry Dataset — October 19, 2026

Contents

1 Project Overview **6**
- 1.1 Project Identity . . . 6
- 1.2 Core Objectives . . . 6
- 1.3 Non-Goals . . . 6

2 System Architecture **7**
- 2.1 High-Level Architecture Diagram . . . 7
- 2.2 Component Summary . . . 8

3 Communication Layer **9**
- 3.1 Protocol Overview . . . 9
- 3.2 Packet Flow — HTTP . . . 10
- 3.3 Packet Schema . . . 10
- 3.4 MQTT Topic Taxonomy & Control Loop . . . 11

4 Security Implementation **12**
- 4.1 Security Layer Architecture . . . 12
- 4.2 Device Registration Flow . . . 12
- 4.3 HMAC-SHA256 Implementation . . . 12
- 4.4 AES-256-GCM Encryption . . . 13
- 4.5 Remote Node Revocation & Session Management . . . 13
- 4.6 Open vs. Secured Traffic Auditing . . . 14
- 4.7 Dynamic Role-Based Access Control & Tier Management . . . 14
- 4.8 Fault Codes . . . 14

5 Vehicle Simulation Engine **15**
- 5.1 Physics Model Overview . . . 15
  - 5.1.1 Load Factor Computation . . . 15
  - 5.1.2 Battery Drain Model . . . 15
  - 5.1.3 Thermal Model . . . 15
  - 5.1.4 Range Estimation . . . 15
- 5.2 Vehicle Operating Modes . . . 16
- 5.3 Physics Interdependency Map . . . 16

6 AI Integration **17**
- 6.1 Architecture . . . 17
- 6.2 Two AI Surfaces . . . 17
- 6.3 Prompt Engineering . . . 17
- 6.4 AI Throttling . . . 18
- 6.5 Concurrency Serialization & Event Loop Safety . . . 18

6.6 Hierarchical Kill-Switch & Graceful Service Degradation 18

7 Business Model & Centralized AI Economics **20**
7.1 The Core Thesis and Economic Rationale 20
7.2 The "Driver Intuition" Fallacy 20
7.3 Industrial Value Proposition for Electric Vehicle Manufacturers 21
7.3.1 Bill-of-Materials (BOM) Cost Breakdown 21
7.3.2 Eliminating Parasitic Battery Drain and Thermal Overhead 22
7.3.3 Fleet-Wide Predictive Maintenance vs. Siloed In-Car Heuristics . . 23
7.4 Cloud vs. On-Vehicle Edge AI: Architectural Lifecycle Comparison 23
7.4.1 Zero-Downtime Model Deployment vs. Risky Automotive ECU OTA Flashing 23
7.4.2 Hardware Longevity and Automotive Lifespan Mismatch 24
7.5 Dual-Tier Commercial SaaS Framework & Unit Economics 24
7.5.1 Tier Breakdown: Community vs. Pro 24
7.5.2 Rigorous Cloud Unit Economics 25
7.6 Engineering Honest Analysis: Gaps, Disadvantages & Architectural Mitigations 26
7.7 Real-World Automotive Industry Case Studies 28
7.8 Scripted Engineering Defense Dialogues 28
7.8.1 Dialogue 1: The "Why Not an In-Car Chip?" Challenge 28
7.8.2 Dialogue 2: The "What If the Internet Disconnects?" Challenge . . 29
7.8.3 Dialogue 3: The "Driver Intuition" Challenge 29

8 Database Schema **30**
8.1 Schema Overview 30
8.2 Packets Table 30
8.3 Devices & Auth Logs Tables 31
8.4 Server Config Keys 32

9 Network Operations Center (NOC) **33**
9.1 NOC Purpose 33
9.2 NOC Terminology Glossary 33
9.3 NOC Live Controls 34
9.4 NOC Packet Inspector Fields 34

10 Frontend Pages **36**
10.1 Routing Architecture 36
10.2 Driver Dashboard (`/driver`) 36
10.3 NOC Dashboard (`/noc`) 37
10.4 Admin Console (`/admin`) 37
10.5 Mobile Views (`/alphamobile`, `/betamobile`, etc.) 37
10.6 Dynamic Route Parameter Normalization 37
10.7 Real-Time Mobile Access Reactivity (MobileAccessGate) 38

11 Chaos & Simulation Middleware **39**
11.1 Purpose 39
11.2 Middleware Pipeline 39
11.3 Tamper Detection Sequence 39

12 API Reference **40**
12.1 Complete Endpoint Inventory 40

13 CCNS Syllabus Mapping **42**

14 Startup & Deployment **44**
14.1 Prerequisites 44
14.2 One-Click Startup 44
14.3 Port Map 44

15 Security Analysis **45**
15.1 Threat Model 45
15.2 Cryptographic Strength 45

A WebSocket Event Types **46**

B Fault Code Reference **47**

C Physics Engine — Key Constants **48**

D Comprehensive Viva Voce Examination Guide & Defense Tactics **49**
D.1 Core Academic Positioning & The Three Golden Rules 49
D.2 CCNS Technical Examination: Questions & Authoritative Answers 49
D.2.1 Unit 1: Data Communication & Application Layer Protocols 49
D.2.2 Unit 2: Network Models & Transport Protocols (HTTP vs. MQTT vs. WebSocket) 50
D.2.3 Unit 3: Transport Layer Reliability & Chaos Engineering Middleware 51
D.2.4 Unit 4: Cryptographic Wire Security & Integrity 51
D.2.5 Unit 5 & 6: Electric Vehicle Industrial Engineering, Gaps & Business Economics 52
D.3 Examiner Trap Questions & Scripted Counter-Tactics 53
D.4 Live Examination Demonstration Playbook 54

List of Tables

2.1 CVIS Component Inventory 8

3.1 Transport Protocol Comparison 9
3.2 Telemetry Payload Schema v1.0 11

4.1 Vehicle Fault Codes 14

5.1 Vehicle Operating Modes — Emergent Classification 16

6.1 AI Endpoints 17

7.1 Detailed Bill-of-Materials (BOM) Comparison: Edge AI vs. CVIS Thin Client 22
7.2 CVIS Commercial Tier Architecture 25
7.3 Comprehensive Engineering Gaps and CVIS Architectural Mitigations . . . 27

8.1 Database Tables 30
8.2 Runtime Server Configuration Keys 32

9.1 NOC Terminology Reference 33
9.2 NOC Live Controls — Backend Effect Mapping 34

10.1 Frontend Routes 36

12.1 Full API Endpoint Reference 40

13.1 CVIS Features Mapped to CCNS Units 42

14.1 Software Prerequisites 44
14.2 Service Port Map 44

15.1 Threat Model and Mitigations 45

A.1 WebSocket Event Schema 46

B.1 Extended Fault Code Reference 47

C.1 Physics Engine Constants 48

List of Figures

2.1 CVIS High-Level System Architecture 7
3.1 HTTP Telemetry Ingest Pipeline 10
4.1 CVIS Security Layer Stack 12
5.1 Physics Variable Interdependency Graph 16
11.1 ChaosMiddleware Execution Pipeline 39

Chapter 1

Project Overview

1.1 Project Identity

The **Connected Vehicle Intelligence System (CVIS)** is an academic prototype built for the *Computer Communication & Network Security (CCNS)* course. Its central thesis is that AI reasoning belongs in the cloud, not embedded in each vehicle. The vehicle is a *thin, securely-networked client*; the laptop running the backend acts as the data-centre. CVIS demonstrates the **secure communication layer** that makes this architecture viable.

Core Thesis

AI lives centrally (cloud/data-centre). The vehicle is a thin client. The project grades the **secure communication pipeline**, not the AI itself.

1.2 Core Objectives

1. Simulate a realistic EV telemetry pipeline from edge device to centralised server.
2. Enforce cryptographic integrity (HMAC-SHA256) and optional confidentiality (AES-256-GCM) on every packet.
3. Demonstrate runtime protocol switching between HTTP REST and MQTT publish-subscribe without restarting any service.
4. Provide a live Network Operations Center (NOC) dashboard that visualises packet flow, security events, and chaos simulations in real time.
5. Integrate Ollama AI reasoning over multi-factor telemetry to produce contextually accurate driver recommendations.

1.3 Non-Goals

- Real EV hardware or real sensors — telemetry is simulated on an ESP32 or via the Python physics engine.
- A generic chatbot with no telemetry grounding.
- Three separate frontend codebases (one unified Next.js app serves all three roles).

Chapter 2

System Architecture

2.1 High-Level Architecture Diagram

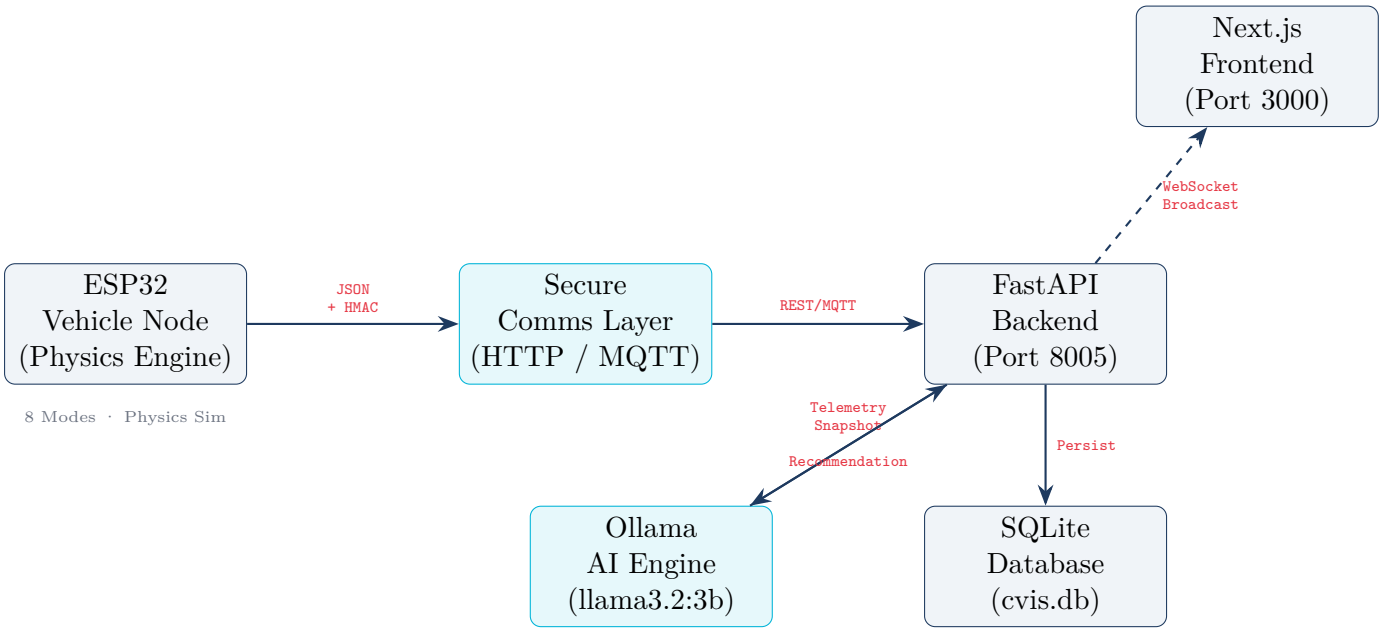


Figure 2.1: CVIS High-Level System Architecture

2.2 Component Summary

Table 2.1: CVIS Component Inventory

Component	Technology	Location	Role
ESP32 Firmware	Arduino C++	`/firmware`	Stateful physics engine; generates telemetry; signs with HMAC; supports HTTP + MQTT
Python Simulator	Python 3	`/backend/simulator.py`	Lateral vehicle physics emulator; same logic as firmware, used for development
FastAPI Backend	Python + FastAPI	`/backend`	Core server: auth, crypto, chaos middleware, AI bridge, WebSocket fan-out
SQLite Database	aiosqlite	`/backend/cvis.db`	Persists packets, telemetry, devices, auth logs, and server config
Ollama AI	llama3.2:3b / phi3-mini	Local Ollama	Multi-factor telemetry reasoning, driver recommendations, free-text chat
Next.js Frontend	Next.js + Tailwind	`/frontend`	Unified three-role UI: Driver, NOC, Admin

Chapter 3

Communication Layer

3.1 Protocol Overview

CVIS supports two fully-functional transport protocols, switchable at runtime with zero restarts:

Table 3.1: Transport Protocol Comparison

Property	**HTTP REST**	**MQTT Pub/Sub**
Pattern	Request-Response	Publish-Subscribe
Connection	Stateless, per-request	Persistent TCP via broker
Topic/Endpoint	`POST /api/v1/telemetry`	`cvis/telemetry/{device_id}`
Auth mechanism	`X-API-Key` header	`_meta.api_key` in envelope
HMAC transport	`X-HMAC-Signature` header	`_meta.hmac` in envelope
Config subscribe	Poll `/api/v1/config/protocol`	Subscribe `cvis/config/#`
Switch latency	Immediate (next packet)	MQTT broker must be running
Best used when	Demonstration, reliability	Bandwidth-constrained, IoT

3.2 Packet Flow — HTTP

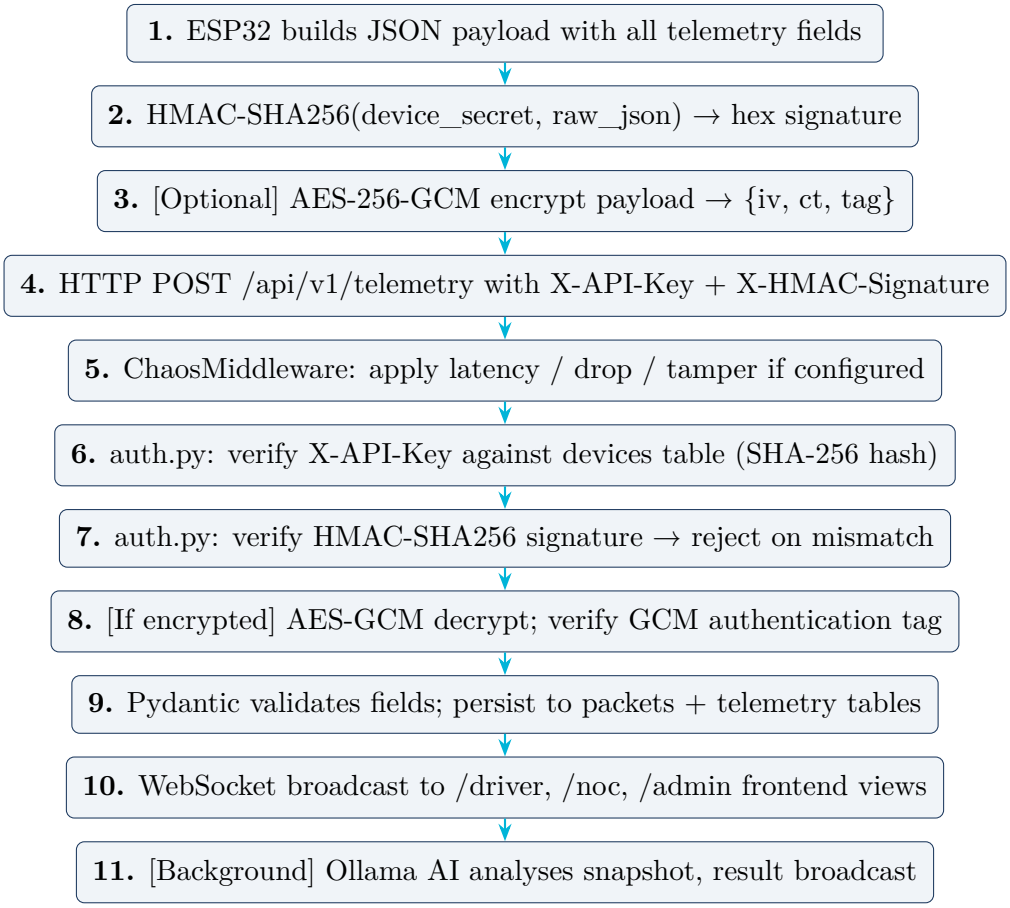


Figure 3.1: HTTP Telemetry Ingest Pipeline

3.3 Packet Schema

Every packet conforms to the following JSON schema (defined in `backend/models.py`):

Table 3.2: Telemetry Payload Schema v1.0

Field	Type	Required	Description & Bounds
`device_id`	string	Yes	Unique vehicle ID (1–64 chars)
`schema_version`	string	No	Schema version, default `"1.0"`
`timestamp_ms`	integer	Yes	ESP32 `millis()` at packet creation
`mode`	string	Yes	One of 8 allowed modes (see Table 5.1)
`speed_kmh`	float	No	Vehicle speed; $0 \leq v \leq 250$ km/h
`battery_pct`	float	No	State of charge; $0 \leq \text{SoC} \leq 100\%$
`battery_temp_c`	float	No	Battery pack temp; $-20 \leq T \leq 100\,^{\circ}\text{C}$
`motor_temp_c`	float	No	Motor temp; $-20 \leq T \leq 150\,^{\circ}\text{C}$
`range_km`	float	No	Estimated remaining range; $0 \leq r \leq 1000$ km
`fault_code`	integer	No	`0x00` = none, `0x02` = overheat, `0x04` = motor
`charging_rate_w`	float	No	Charging power; 0 when not charging
Advanced Physics Fields (Optional — Schema v2.3)			
`ambient_temp_c`	float	No	Outside air temperature in °C
`headwind_kmh`	float	No	Headwind speed in km/h
`road_gradient_pct`	float	No	Road gradient (+uphill / -downhill)
`tire_pressure_psi`	float	No	Average tire pressure in PSI
`cabin_climate_w`	float	No	AC/Heater power draw in Watts
`max_cell_voltage_delta`	float	No	Max voltage difference between cells (V)

3.4 MQTT Topic Taxonomy & Control Loop

In addition to inbound telemetry streaming, the MQTT communication layer coordinates bidirectional configuration and node revocation:

- `cvis/telemetry/{device_id}`: Uplink topic for vehicle telemetry envelopes (plaintext or AES-256-GCM encrypted).
- `cvis/config/protocol`: Downlink broadcast topic where the backend announces runtime transport shifts (`"http"` vs `"mqtt"`). Subscribed firmware nodes switch protocols on receipt without power cycling.
- `cvis/config/encryption`: Downlink broadcast topic toggling AES-256-GCM encryption mode on or off.
- `cvis/config/control/{device_id}`: Unicast node control channel. When the NOC deactivates a node, a `"disconnect"` message is published to this topic. Subscribed vehicle firmware immediately transitions into an idling suspended state, halting all radio transmissions until a `"reconnect"` instruction is received.

On resource-constrained microcontrollers (ESP8266/ESP32), JSON memory buffers are strictly calibrated to `StaticJsonDocument<1024>` to ensure full 17-parameter multi-factor telemetry frames deserialize without memory exhaustion or silent key dropping.

Chapter 4

Security Implementation

4.1 Security Layer Architecture

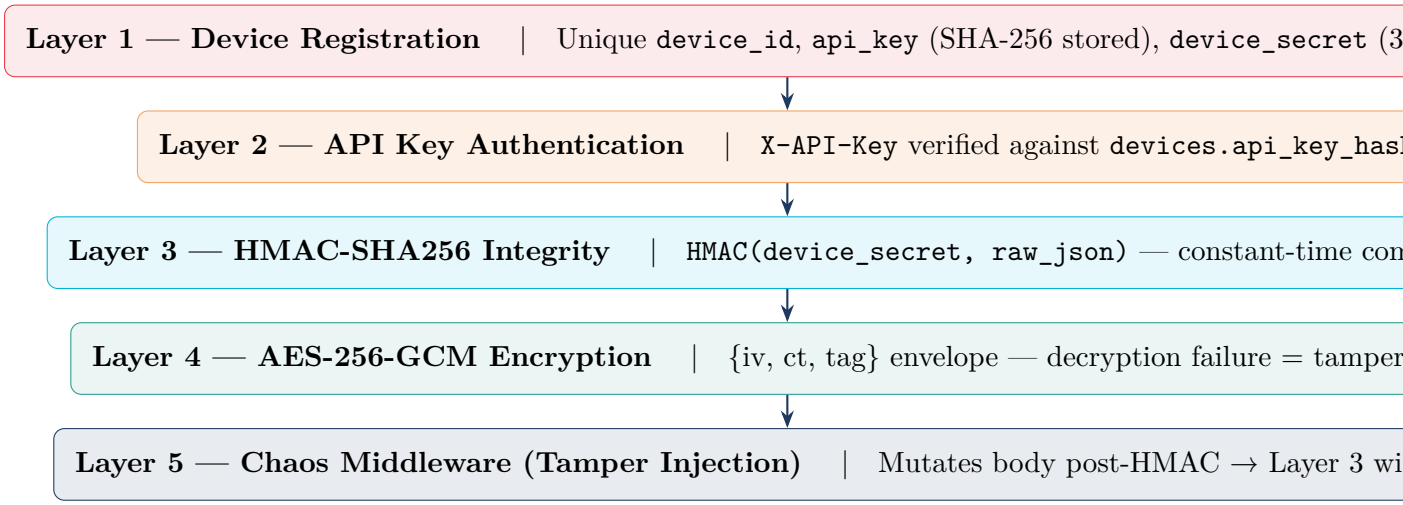


Figure 4.1: CVIS Security Layer Stack

4.2 Device Registration Flow

When the Python simulator or an ESP32 first connects:

1. `POST /api/v1/devices/register` with `{"device_id": "SIM-001"}`.
2. Backend generates: `api_key = secrets.token_hex(32)` and `device_secret = secrets.token_hex(32)`.
3. `api_key_hash = SHA256(api_key)` is stored in `devices` table. The plaintext `api_key` is returned once.
4. All subsequent requests include `X-API-Key: <plaintext>` and `X-HMAC-Signature: <hex>`.

4.3 HMAC-SHA256 Implementation

HMAC is computed over the *exact raw JSON bytes* as transmitted. This is critical: even a single extra space in the JSON would change the HMAC and trigger rejection.

Listing 4.1: HMAC Computation (backend/crypto.py)

```python
def compute_hmac(secret_hex: str, payload: str) -> str:
    key = bytes.fromhex(secret_hex)        # 32-byte (256-bit) key
    sig = hmac.new(key, payload.encode("utf-8"), hashlib.sha256)
    return sig.hexdigest()                 # 64-char lowercase hex

def verify_hmac(secret_hex: str, payload: str, signature_hex: str) ->
    bool:
    expected = compute_hmac(secret_hex, payload)
    # hmac.compare_digest prevents timing-oracle attacks
    return hmac.compare_digest(expected, signature_hex.strip().lower
        ())
```

4.4 AES-256-GCM Encryption

Listing 4.2: AES-GCM Encrypt/Decrypt (backend/crypto.py)

```python
def aes_gcm_encrypt(secret_hex: str, plaintext: str) -> dict:
    key    = bytes.fromhex(secret_hex)   # 32 bytes = 256-bit AES key
    aesgcm = AESGCM(key)
    nonce  = os.urandom(12)              # 96-bit random nonce per
        message
    ct_with_tag = aesgcm.encrypt(nonce, plaintext.encode("utf-8"),
        None)
    ct  = ct_with_tag[:-16]              # ciphertext bytes
    tag = ct_with_tag[-16:]              # 128-bit GCM authentication
        tag
    return {"iv": nonce.hex(), "ct": ct.hex(), "tag": tag.hex()}
```

The GCM **authentication tag** doubles as an integrity check: decrypting with a mismatched key or tampered ciphertext raises `InvalidTag`, which the backend logs as a `tamper_detected` event.

4.5 Remote Node Revocation & Session Management

To emulate real-world vehicular certificate and session revocation without tearing down underlying TCP sockets, CVIS implements centralized node deactivation:

1. **Revocation Trigger**: An operator at the NOC issues a disconnect command (`POST /api/v1/control/disconnect`).
2. **Database State Transition**: The backend sets `devices.active = 0` and records an audit entry in `auth_logs` under event type `disconnected`.
3. **Protocol-Specific Enforcement**:
   - *HTTP*: Subsequent telemetry requests from the revoked device are rejected with HTTP 401 (*Unauthorized*) at the API key authentication gate.
   - *MQTT*: A downlink revocation signal is published to `cvis/config/control/{device_id}`. The firmware halts telemetry loop transmissions, preventing broker congestion.

4. **Re-activation**: When connectivity is re-authorized (`POST /api/v1/control/reconnect`), the device's active flag is restored, a `reconnected` audit log is generated, and resume instructions are dispatched to the node.

4.6 Open vs. Secured Traffic Auditing

For academic and demonstration purposes, authentication enforcement can be toggled live via the NOC (`POST /api/v1/config/auth`). When disabled, incoming frames without valid API keys or HMAC signatures are accepted rather than dropped. However, to maintain strict security auditing:

- Packets are saved to SQLite with `auth_status = 'no_auth'` rather than `'ok'`.
- A dedicated entry is written to `auth_logs` indicating unauthenticated access.
- The WebSocket broadcast flags the packet with `status = 'ok'` and `auth_status = 'no_auth'`, enabling the NOC packet inspector to highlight open traffic clearly.

4.7 Dynamic Role-Based Access Control & Tier Management

Vehicle identities are segmented into distinct capability tiers:

- **Free Tier**: Access to standard telemetry streaming, physical health gauges, and speed metrics. Advanced neural chat and mobile applications are gated.
- **Premium Tier**: Unlocks central AI reasoning, real-time efficiency scoring, predictive coaching, and dedicated mobile cockpit views (`/alphamobile`, `/betamobile`).

Tier state is persisted in SQLite (`devices.tier` and `devices.mobile_access_enabled`). When a driver upgrades in the dashboard, the backend updates the database and broadcasts a `mobile_access_changed` event via WebSocket. Frontend gate components (`MobileAccessGate`) consume this event in real time to unlock mobile views instantaneously without requiring browser reloads.

4.8 Fault Codes

Table 4.1: Vehicle Fault Codes

Code (hex)	Name	Description & Physics Trigger
`0x00`	No Fault	Normal operation
`0x02`	Battery Overheating	Battery temp $> 60\,^{\circ}\mathrm{C}$ — caused by high discharge rate under load
`0x04`	Motor Fault	Motor temp $> 95\,^{\circ}\mathrm{C}$ — caused by high speed + uphill gradient + drag

Chapter 5

Vehicle Simulation Engine

5.1 Physics Model Overview

The simulator is a **stateful physics engine**. Each tick (every 2 seconds) computes the next state based on the current state. No values are hardcoded per mode; modes are *emergent* from the physics.

5.1.1 Load Factor Computation

The **Load Factor** (L) is the normalised measure of how hard the vehicle is working at any instant:

$$L = \underbrace{\frac{v}{50}}_{\text{speed drag}} + \underbrace{\frac{(v+w)^2}{10000}}_{\text{aerodynamic drag}} + \underbrace{g \cdot 0.5}_{\text{gravity drag}} + \underbrace{\max\left(0,\ \frac{36 - p_t}{10}\right)}_{\text{tire drag}} \tag{5.1}$$

where v = speed (km/h), w = headwind (km/h), g = road gradient (%), p_t = tire pressure (PSI).

5.1.2 Battery Drain Model

$$\Delta\text{SoC} = \left(L \cdot 0.1 + \frac{P_{\text{cabin}}}{5000}\right) \cdot \Delta t \tag{5.2}$$

5.1.3 Thermal Model

Motor Temperature:

$$\Delta T_{\text{motor}} = (L \cdot 2.0 - (T_{\text{motor}} - T_{\text{ambient}}) \cdot 0.05) \cdot \Delta t \tag{5.3}$$

Battery Temperature:

$$\Delta T_{\text{battery}} = (\Delta\text{SoC} \cdot 5.0 - (T_{\text{battery}} - T_{\text{ambient}}) \cdot 0.02) \cdot \Delta t \tag{5.4}$$

5.1.4 Range Estimation

$$r = \text{SoC} \times 3.0 \times f_{\text{tire}} \times f_{\text{wind}} \times f_{\text{cabin}} \tag{5.5}$$

where penalty factors are: $f_{\text{tire}} = 0.9$ if $p_t < 32$ PSI; $f_{\text{wind}} = 0.85$ if $w > 20$ km/h; $f_{\text{cabin}} = 0.95$ if $P_{\text{cabin}} > 1000$ W.

5.2 Vehicle Operating Modes

Table 5.1: Vehicle Operating Modes — Emergent Classification

Mode	Classification Trigger	Typical Characteristics
Healthy	$30 \leq v \leq 80$ km/h	Normal cruise; moderate load
Eco	(button-selected / low speed)	Conservative speed, max range
Sport	$v > 80$ km/h	High motor temp, fast drain
Heavy Traffic	$0 < v < 30$ km/h	Stop-start, cabin AC dominant
Low Battery	SoC $\leq 15\%$	Speed limited; urgent AI alert
Battery Overheating	$T_{\text{batt}} > 60°\text{C}$	Fault 0x02; limp mode (40 km/h cap)
Charging	$v = 0$ and $P_{\text{charge}} > 0$	Battery replenishing
Motor Fault	$T_{\text{motor}} > 95°\text{C}$	Fault 0x04; forced stop

5.3 Physics Interdependency Map

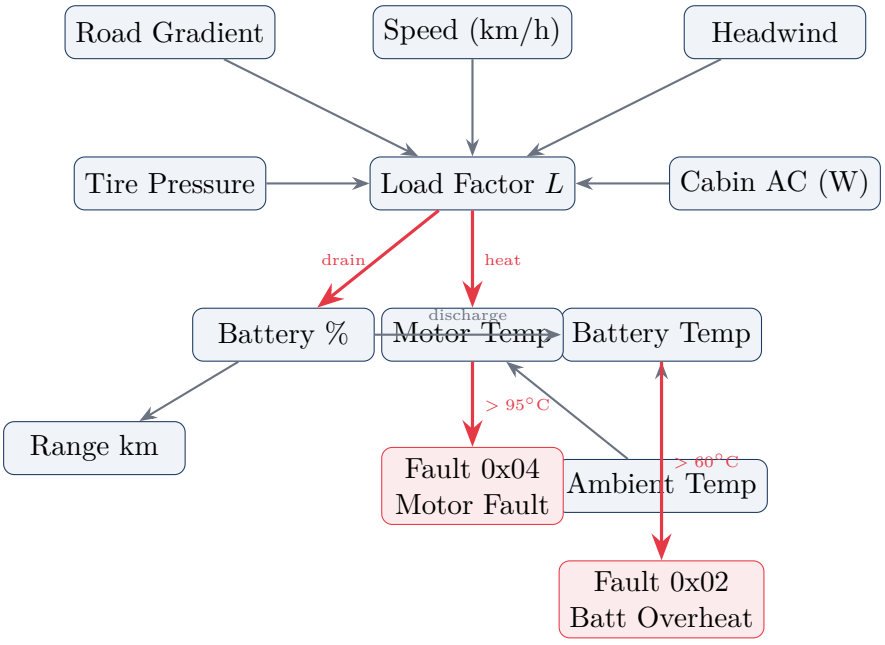


Figure 5.1: Physics Variable Interdependency Graph

Chapter 6

AI Integration

6.1 Architecture

CVIS uses **Ollama** running locally with `llama3.2:3b` (or `phi3-mini`). The AI is called via HTTP to `http://localhost:11434/api/chat`. The model is never called directly from the frontend — it is always mediated by the FastAPI backend, which ensures telemetry grounding.

6.2 Two AI Surfaces

Table 6.1: AI Endpoints

Surface	Endpoint	Behaviour
Auto Recommendation	Background task per telemetry update	Analyses combined snapshot; produces severity-tagged alert; broadcast via WebSocket
Driver Chat	`POST /api/v1/ai/chat`	Free-text Q&A grounded in current + last 10 readings; cannot invent vehicle state

6.3 Prompt Engineering

The prompt is built in `backend/ai/prompt_builder.py`. The user message sent to Ollama contains:

1. **Current Snapshot**: all 17 telemetry fields from the latest packet.
2. **Advanced Physics Section**: ambient temp, headwind, road gradient, tire pressure, cabin load, cell voltage delta — labelled `=== ADVANCED PHYSICS ===`.
3. **Trend Block**: last 5 readings rendered as a sparkline text table (battery %, speed, motor temp, mode).
4. **Range Violations**: any field that has crossed its warning threshold.
5. **Mode Transition**: previous mode is explicitly shown when a mode change is detected.

The system prompt instructs the model to: (1) classify severity as `NORMAL` / `CAUTION` / `WARNING` / `CRITICAL`, (2) reason from the *combination* of factors rather than any single threshold, and (3) output exactly two lines: severity label + recommendation.

6.4 AI Throttling

To prevent LLM overload, the background AI task fires only when:

- At least 20 seconds have elapsed since the last inference, **OR**
- The vehicle mode has changed since the last packet.

6.5 Concurrency Serialization & Event Loop Safety

High-frequency telemetry ingests across multi-vehicle fleets can trigger simultaneous inference requests to the local Ollama daemon. Without concurrency bounding, parallel execution of 3B parameter models on consumer hardware causes severe memory contention and latency spikes.

In `backend/ai/ollama_client.py`, inferences are strictly serialized through an asynchronous lock. To prevent the notorious Python `RuntimeError: Task attached to a different loop` issue—which occurs when an `asyncio.Lock` is instantiated at module import time before FastAPI initializes its runtime event loop—CVIS employs a thread-safe lazy factory:

Listing 6.1: Lazy Event-Loop Lock Factory (backend/ai/ollama_client.py)

```
_ollama_lock: Optional[asyncio.Lock] = None

def _get_ollama_lock() -> asyncio.Lock:
    global _ollama_lock
    if _ollama_lock is None:
        _ollama_lock = asyncio.Lock()
    return _ollama_lock
```

Every inference invocation in `generate_recommendation()` and `chat_with_grounding()` executes inside `async with _get_ollama_lock():`, guaranteeing sequential model execution bound safely to the active running loop.

6.6 Hierarchical Kill-Switch & Graceful Service Degradation

CVIS enforces a two-tiered administrative kill-switch hierarchy:

1. **Global Service Switch** (`server_config.ai_service_enabled`): Toggled via `POST /api/v1/control/ai-service`. When disabled by NOC operators, all automated background inferences and conversational AI queries across the entire fleet are instantly halted.
2. **Per-Vehicle Gating** (`vehicle_ai_enabled`): Toggled via `POST /api/v1/control/ai-service/vehicle`. Allows operators to selectively disable AI reasoning for individual vehicles (e.g., degraded nodes or billing holds). The global kill-switch maintains strict precedence over per-vehicle settings.

When AI is toggled off or if the local Ollama process is unresponsive, the system gracefully degrades: the background pipeline suppresses inference without dropping

telemetry, and the driver chat endpoint (`POST /api/v1/ai/chat`) immediately returns an HTTP `503 Service Unavailable` status with an explicit status payload:

Listing 6.2: HTTP 503 AI Degradation Response

```
{
  "status": "disabled",
  "detail": "AI service is currently disabled by administrator."
}
```

The frontend detects this HTTP 503 code and renders an unobtrusive service pause badge rather than spinning indefinitely.

Chapter 7

Business Model & Centralized AI Economics

7.1 The Core Thesis and Economic Rationale

The fundamental architectural thesis of the **Connected Vehicle Intelligence System (CVIS)** is:

> **High-level artificial intelligence reasoning, telematics synthesis, and predictive diagnostics belong centrally in the cloud and data center—not embedded per vehicle. The vehicle must operate strictly as a thin, hardened, and securely-networked edge client.**

In contemporary automotive software engineering, an active debate centers on vehicle compute topology: *Should an electric vehicle (EV) function as an isolated supercomputer on wheels with massive on-board silicon, or should it operate as a thin client backed by centralized cloud intelligence?*

CVIS demonstrates that for **high-level multi-factor telemetry reasoning, cell degradation forecasting, driver coaching, and fleet management**, centralized cloud intelligence is overwhelmingly superior to on-vehicle edge computing in:

1. **Manufacturing Bill-of-Materials (BOM) Costs** ($1,700–$3,500 edge AI SoC vs. $37–$92 thin gateway);
2. **Parasitic Battery Drain and Thermal Efficiency** (eliminating 70W–250W phantom sentry drain);
3. **Fleet-Wide Swarm Intelligence** (detecting battery batch defects across millions of vehicles);
4. **Model Update Agility and Safety** (30-second zero-downtime cloud hot-swapping vs. high-risk multi-gigabyte ECU flashing over the air);
5. **Hardware Lifespan Alignment** (preventing vehicle obsolescence over a 12–15 year automotive operational lifecycle).

7.2 The "Driver Intuition" Fallacy

A frequent industry objection to connected vehicle AI systems is the *Driver Intuition Fallacy*:

> *"Why should an operator pay for centralized AI when an experienced driver intuitively knows that high speeds and steep hills drain the battery and heat up the motor?"*

While human intuition is capable of learning crude, qualitative heuristics (e.g., "driving faster drains the battery"), human cognition possesses severe and fatal limitations in modern electric vehicles:

1. **Reactive vs. Predictive Anomaly Detection**: Human drivers react exclusively to dashboard warning indicators, which illuminate only *after* a critical threshold is breached (e.g., Battery Temp $> 60°$C or Motor Temp $> 95°$C). At this point, irreversible cell degradation or forced vehicle immobilization has already begun. CVIS utilizes **Sliding-Window Trend Analysis** across consecutive telemetry packets to compute rate-of-change derivatives ($\Delta T/\Delta t$). The cloud AI detects thermal gradient anomalies rising 1.5°C/min faster than expected under current load, alerting the operator up to 15 minutes *before* any hardware threshold is tripped.

2. **Coupled Multi-Variate Physics vs. Gut Feeling**: EV range and thermal behavior are governed by non-linear coupled differential equations combining aerodynamics, road gradient, tire pressure, HVAC auxiliary draw, and cell internal resistance. If a driver asks:

   "I have 24% battery remaining, ambient temperature is 38°C, headwind is 35 km/h, and I need to climb a 6% grade in stop-and-go traffic for 18 km with air conditioning active—will I reach the destination?"

   Human intuition is mathematically incapable of answering this. CVIS acts as a real-time **Conversational Physics Simulator**, integrating all 17 live telemetry values against validated physical load models to provide a mathematically bounded, high-confidence guidance response.

3. **Fleet-Wide Swarm Intelligence**: An individual driver is isolated within their own vehicle envelope. Centralized AI aggregates telemetry across thousands of deployed nodes in real time. If 20 vehicles along a specific highway corridor experience anomalous battery heating or localized rolling-resistance spikes due to extreme ambient heat or road conditions, the central AI immediately flags predictive alerts to all upstream vehicles entering that corridor.

7.3 Industrial Value Proposition for Electric Vehicle Manufacturers

7.3.1 Bill-of-Materials (BOM) Cost Breakdown

In high-volume electric vehicle manufacturing (e.g., 500,000 to 2,000,000 units annually), saving even $50 per vehicle directly translates to tens of millions of dollars in net margin. Table 7.1 details the exact component-level cost differential between an edge-compute vehicle architecture and the CVIS thin-client gateway approach.

Table 7.1: Detailed Bill-of-Materials (BOM) Comparison: Edge AI vs. CVIS Thin Client

Subsystem	Dedicated Edge AI Node	CVIS Gateway Architecture	Savings per Car
Processing Silicon	Automotive AI SoC (NVIDIA DRIVE Orin / Snapdragon Ride): $1,200–$2,500	Thin Automotive MCU / Gateway (ESP32 / NXP S32G / STM32): $15–$45	-$1,185 to -$2,455
High-Speed DRAM	16GB–32GB LPDDR5 ECC Automotive DRAM: $180–$350	16MB–64MB embedded Flash / SRAM: $5–$12	-$175 to -$338
Thermal Management	Cold-plate liquid cooling loop, hoses, connectors: $150–$300	Passive convection heatsink casing: $4–$10	-$146 to -$290
Safety Qualification	ISO 26262 ASIL-D & AEC-Q100 for 3nm/5nm silicon: $80–$150	Proven mature automotive qualification: $5–$10	-$75 to -$140
Power Supply & Wire	High-amperage 12V/48V step-down DC converter (150W–350W): $120–$220	Standard low-voltage auxiliary feed (<5W): $8–$15	-$112 to -$205
TOTAL HARDWARE BOM	**$1,735 – $3,520**	**$37 – $92**	**$1,698 – $3,428**

Industrial Financial Impact for 500,000 Annual EV Volume

$$\text{Annual Net Hardware Cost Savings} = 500,000 \times \$2,000 = \$\mathbf{1,000,000,000}\ (\mathbf{1}\ \text{Billion USD})$$

By eliminating redundant edge computing silicon, an automotive OEM can either lower retail pricing to dominate competitive market segments or reinvest capital into cell chemistry and motor stator efficiency.

7.3.2 Eliminating Parasitic Battery Drain and Thermal Overhead

Electric vehicle range anxiety remains a decisive consumer barrier. In-car edge AI computers impose a severe, continuous parasitic electrical load:

- **Continuous Power Draw**: Running multi-billion parameter neural networks locally on automotive chips consumes **70W to 250W** of continuous high-voltage power.
- **Thermal System Congestion**: On hot summer days (35°C–45°C ambient), the Battery Thermal Management System (BTMS) is already operating near capacity cooling the traction battery and inverter. Adding a 200W heat source to

the coolant circuit forces the A/C compressor to cycle harder, accelerating range degradation.

- **Phantom Sentry Loss**: In parked standby mode, an edge computer drawing 150W consumes **3.6 kWh per day**—resulting in **20 to 30 km (12–18 miles) of battery range lost every 24 hours** without moving.
- **The CVIS Advantage**: In CVIS, the vehicle's telemetry gateway draws under **1.5 Watts**. All heavy matrix multiplications and language synthesis occur in cloud data centers powered by industrial electrical grids. The vehicle's high-voltage pack is preserved exclusively for motion and climate control.

7.3.3 Fleet-Wide Predictive Maintenance vs. Siloed In-Car Heuristics

When AI is confined to an isolated vehicle:

1. It possesses zero knowledge beyond its own sensors.
2. If cell #42 in a battery module exhibits a subtle voltage delta during high-rate fast charging, the local computer has no statistical baseline to determine whether this represents normal cell aging or an imminent thermal runaway hazard.

In the **CVIS Centralized Cloud Model**:

1. Telemetry from millions of deployed vehicles streams into a central high-speed ingest database (`packets` and `telemetry`).
2. Cloud analytics execute **Cross-Fleet Anomaly Clustering**:
   - If 14 vehicles equipped with *Batch 2024-Q2 CATL LFP cells* exhibit a 0.04V voltage variance during fast charging between 70% and 80% SoC below 10°C, the cloud AI identifies a **fleet-wide manufacturing batch defect**.
   - The OEM can push an immediate targeted charging curve limit to those exact VINs before a single battery pack catches fire on a public highway or triggers a multi-billion dollar safety recall.
3. **Continuous Swarm Learning**: When vehicles encountering mountain passes report sudden headwind and freezing temperatures that increase consumption from 160 Wh/km to 240 Wh/km, the cloud model immediately updates range prediction models for all subsequent vehicles routed along that corridor.

7.4 Cloud vs. On-Vehicle Edge AI: Architectural Lifecycle Comparison

7.4.1 Zero-Downtime Model Deployment vs. Risky Automotive ECU OTA Flashing

Automotive Over-the-Air (OTA) firmware updates are notoriously high-risk, expensive, and logistically fragile:

- **The Edge AI Dilemma**: Updating a local model weights file (e.g., 3GB to 8GB) requires broadcasting multi-gigabyte files to hundreds of thousands of cars over cellular 4G/5G connections. Automakers pay millions in telecom carrier data egress fees. The driver must park the car for 30–60 minutes. If power cuts or flash memory corrupts mid-flash, the vehicle ECU is **bricked**, requiring dealer towing. Furthermore, flashing safety ECUs requires lengthy regulatory homologation re-certification.
- **The CVIS Cloud Paradigm**: Model upgrades occur **100% in the data center** via dynamic model hot-swapping (`POST /api/v1/ai/models/switch`) or blue-green container deployments. Upgrading from `llama3.2:3b` to `llama3.3:8b` or `qwen3:8b` takes **under 30 seconds**. There is **zero vehicle downtime**, **zero cellular OTA download risk**, and **instant 1-second rollback** if regressions appear.

7.4.2 Hardware Longevity and Automotive Lifespan Mismatch

The operational lifespan of a modern passenger vehicle is **12 to 15 years**.

- **The Edge Silicon Trap**: If an automaker integrates an AI processor in 2024, by 2028 that chip is four hardware generations obsolete. It lacks the TOPS, memory bandwidth, and quantization support to run 2030 foundation models. Within five years, the vehicle's onboard assistant feels sluggish and abandoned.
- **The CVIS Future-Proof Guarantee**: Because reasoning lives in the cloud, a 10-year-old vehicle built in 2024 continues to receive state-of-the-art multi-factor intelligence in 2034. When the data center upgrades its GPU clusters from RTX 4060s to Blackwell or optical AI silicon, every vehicle in the fleet immediately inherits that compute capability.

7.5 Dual-Tier Commercial SaaS Framework & Unit Economics

7.5.1 Tier Breakdown: Community vs. Pro

Table 7.2 outlines the CVIS dual-tier SaaS subscription architecture.

Table 7.2: CVIS Commercial Tier Architecture

Tier	Market & Pricing	Included Features & SLA
CVIS Community (Free Tier)	Included with every standard EV purchase; small fleets. **Price: $0 / month**	• Standard telemetry ingestion (HMAC-SHA256 authenticated) • Standard Driver Dashboard (`/driver`) • 7-day rolling telemetry persistence • Pre-warmed cached AI insights (rate-limited) • OEM Benefit: Fleet-wide battery training telemetry
CVIS Fleet Pro (Premium Tier)	Commercial logistics fleets, ride-share networks, tech-forward owners. **Price: $9.99 / car / month** ($99 / car / year)	• Full AEAD AES-256-GCM wire encryption • Real-Time NOC Dashboard (`/noc`) with live controls • Real-time multi-factor neural reasoning (<500 ms SLA) • Conversational Driver Chat with Neural Voice Synthesis (TTS) • Predictive cell degradation & impedance modeling • Replay attack mitigation with monotonic nonce windows • 24-month compliance telemetry retention & CSV export

7.5.2 Rigorous Cloud Unit Economics

Let us evaluate the exact cloud operational costs for an OEM serving **100,000 active Pro subscribers**:

1. **Ingestion & Database Infrastructure**: 100,000 vehicles sending 900-byte packets every 3 seconds generate $\approx 33,333$ packets/second. Four 32-core server instances running async ASGI workers cost **$1,600 / month**. Clustered distributed time-series database storage costs **$3,200 / month**.

2. **GPU AI Inference Costs**: Telemetry changes within standard bounds are validated by the high-speed cache (<5ms, zero GPU cost). Multi-factor neural completions trigger on state changes and driver queries, averaging 12 completions per car per driving hour. Utilizing continuous batching (vLLM / TensorRT-LLM) on clustered NVIDIA L40S or H100 GPUs (generating 1,200 completions/min per GPU), serving 100,000 cars during peak hours requires ≈ 25 GPUs = $\$\mathbf{19,800}$/month.

3. **Cellular Ingress Overhead**: Telecom bulk cellular ingress contracts: **$15,000 / month**.

Monthly Financial Ledger & Profit Margins

$$\text{Total Cloud Cost} = \$1,600 + \$3,200 + \$19,800 + \$15,000 = \$\mathbf{39,600}/\text{month}$$

$$\text{Cost per Vehicle per Month} = \frac{\$39,600}{100,000} = \$\mathbf{0.396}\ (\sim \mathbf{40}\ \text{cents / month})$$

$$\text{Monthly Subscription Revenue} = 100,000 \times \$9.99 = \$\mathbf{999,000}/\text{month}$$

$$\text{Net Gross Profit} = \$999,000 - \$39,600 = \$\mathbf{959,400}/\text{month} \quad (\mathbf{96.0}\%\ \text{Gross Margin})$$

7.6 Engineering Honest Analysis: Gaps, Disadvantages & Architectural Mitigations

In a rigorous academic defense, declaring a distributed system "flawless" destroys technical credibility. Real engineering requires identifying vulnerabilities and designing deterministic mitigations. Table 7.3 details the five primary vulnerabilities of the centralized cloud model and how CVIS solves each.

Table 7.3: Comprehensive Engineering Gaps and CVIS Architectural Mitigations

Vulnerability / Gap	Real-World Risk	CVIS Architectural Mitigation
1. Cellular Dead Zones & Tunnels	Vehicle enters mountain pass or deep underground garage with 0 bars. Does the car freeze or lose advisory guidance?	**Decoupled Actuation + Pre-Warmed Cache**: Mechanical safety (ABS, ESC, motor torque) operates on hard real-time CAN buses. AI is advisory. The gateway includes a deterministic multi-factor cache (`ai_cache.py`) that answers in <5ms locally. Circular flash buffers store telemetry and drain upon reconnect.
2. Mobile Network Latency & Jitter	4G/5G mobile RAN experiences packet jitter and variable RTT (40ms–400ms).	**MQTT Sub-35ms Transport + WebSockets**: CVIS incorporates runtime-switchable MQTT with compact binary/topic headers, cutting RTT to <35ms. WebSockets push updates down immediately. Human cognitive reaction time is 250–400ms, making 150ms cloud advisory instantaneous.
3. Cellular Ingress Data Volume	1,000,000 cars transmitting 1KB/sec generate 30TB/day, risking massive telco bills.	**Adaptive Telemetry Intervals**: Steady highway cruise transmits every 5–8s. Thermal anomalies or fault triggers accelerate to 1-second bursts. Steady-state data sends 40-byte heartbeats, cutting data volume by 70%+.
4. Cloud Data Center Outages / DDoS	Central data center loses power or suffers DDoS. Does the national fleet become disabled?	**Independent Vehicle Autonomy**: The car is 100% mechanically and electrically drivable without the cloud. Verified via the NOC "Stop AI Service" toggle. In production, multi-region Anycast clusters ensure geo-redundancy.
5. Man-in-the-Middle & Packet Spoofing	Wireless telemetry over cellular is exposed to eavesdropping, packet alteration, and replay attacks.	**Cryptographic Authentication & AEAD**: Every packet is authenticated via HMAC-SHA256 constant-time verification or AES-256-GCM AEAD encryption. Monotonic timestamp nonces reject replayed packets.

7.7 Real-World Automotive Industry Case Studies

The CVIS architectural model directly mirrors the implementations used by leading automotive manufacturers:

1. **Tesla Inc. (Fleet Learning & Shadow Mode)**: A common public misconception is that Tesla vehicles train neural networks inside the car while driving. **They do not.** The onboard FSD computer executes lightweight compiled forward-pass vision inference. When disengagements or edge cases occur, the car compresses telemetry and video snapshots and uploads them over cellular to **Tesla's central Dojo and Cortex GPU supercomputers**. All fleet learning, model retraining, and validation happen centrally, with optimized model weights deployed back to the fleet.

2. **Rivian Automotive (AWS FleetOS)**: Rivian's R1T and R1S vehicles incorporate a custom Telematics Gateway Unit (TGU) communicating with AWS cloud clusters. Battery degradation models, predictive air-suspension damping based on road roughness maps, and motor thermal limits are computed centrally in cloud infrastructure and streamed to vehicles.

3. **NIO (Battery-as-a-Service BaaS)**: Over 2,000 automated NIO battery-swap stations communicate with the NIO Cloud. Connected cars stream cell impedance and thermal curves centrally. When an EV approaches a swap station, the cloud has already determined whether that pack requires re-balancing, slow charging, or cell replacement before it is loaded into another car.

7.8 Scripted Engineering Defense Dialogues

7.8.1 Dialogue 1: The "Why Not an In-Car Chip?" Challenge

Examiner Interaction: Edge Silicon vs. Cloud AI

Examiner: *"Why shouldn't automakers just put a powerful AI chip inside the car? Silicon gets cheaper every year."*

Candidate Defense: *"That sounds attractive in theory, but in real automotive manufacturing it encounters the* ***Automotive Hardware Lifespan Mismatch****. An electric vehicle is engineered to remain operational for 12 to 15 years. AI algorithms and model architectures, however, evolve every 12 to 18 months. If an automaker puts an AI chip in a car in 2024, by 2028 that chip is already obsolete—it cannot run the larger reasoning models or newer attention mechanisms of the future.*

Furthermore, in automotive grade electronics (AEC-Q100 / ISO 26262), silicon qualification takes 3 years before production, meaning the chip in a brand new car is already 3 years old on day one! By making the car a thin, securely-networked client and placing the AI centrally, the automaker can upgrade their data center GPUs from RTX 4060s to H100s or Blackwell tomorrow, and a 10-year-old vehicle in the field instantly inherits state-of-the-art intelligence without requiring a physical recall, an expensive chip replacement, or risking a bricked ECU over an OTA flash."

7.8.2 Dialogue 2: The "What If the Internet Disconnects?" Challenge

Examiner Interaction: Network Outages and Safety

Examiner: *"What happens when the car enters an underground parking lot or a remote highway with zero cellular service? Does the car stop working?"*

Candidate Defense: *"Absolutely not. In CVIS, we maintain a strict architectural separation between* ***Real-Time Vehicle Safety Controls*** *and* ***Supervisory AI Intelligence****. The physical actuators—braking, steering, traction control, motor inverter control—run on hard real-time local microcontrollers over isolated CAN buses. The AI in CVIS is strictly an advisory, predictive, and analytical layer.*

To guarantee continuous operation during network blackouts, CVIS incorporates a local pre-warmed multi-factor intelligence engine (`ai_cache.py`). If cellular connectivity drops, the gateway falls back to this deterministic cache in under 5 milliseconds, providing accurate safety advice and mode recommendations based on live physics rules until the network restores. Simultaneously, telemetry is buffered locally in flash and uploaded upon reconnection. The car never stops, and the driver is never left without guidance."

7.8.3 Dialogue 3: The "Driver Intuition" Challenge

Examiner Interaction: Human Intuition vs. AI Physics Simulation

Examiner: *"Why do we need an AI at all? Doesn't a driver already know that driving fast in cold weather reduces range?"*

Candidate Defense: *"A human driver has basic qualitative intuition, but human intuition cannot solve non-linear multi-variate differential calculus while driving at 100 km/h. For example: If a driver has 22% battery, ambient temperature is -2° C, cabin heater is drawing 2.5 kW, headwind is 25 km/h, and they face a 6% uphill gradient over the next 15 km in heavy traffic—will they make it to the charger?*

Human intuition will guess and risk stranding the vehicle. The CVIS multi-factor AI reasons over all 17 telemetry parameters simultaneously: battery cell delta, internal resistance, motor thermal rise rate, and aerodynamic draw. It computes an exact, mathematically grounded power trajectory and advises the driver: 'Reduce cabin heat to 1 kW and cap cruising speed at 85 km/h to arrive at the charger with 4% reserve.' That is predictive precision that human intuition cannot match."

Chapter 8

Database Schema

8.1 Schema Overview

All data is persisted in a single SQLite file (`backend/cvis.db`) using `aiosqlite` for non-blocking async access. The schema uses WAL (Write-Ahead Logging) for concurrent read-write.

Table 8.1: Database Tables

Table	Primary Key	Purpose
`packets`	`id` (auto)	Raw log of every inbound packet; protocol-agnostic
`telemetry`	`id` (auto)	Parsed, structured telemetry fields per packet
`devices`	`device_id`	Registered vehicle nodes with API key hash and secret
`auth_logs`	`id` (auto)	Every auth and tamper event
`server_config`	`key`	Key-value store for all runtime configuration
`schema_version`	`version`	Migration history tracker

8.2 Packets Table

Listing 8.1: Packets Table DDL

```
CREATE TABLE packets (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    received_at       TEXT    NOT NULL,
    device_id         TEXT    NOT NULL,
    protocol          TEXT    NOT NULL DEFAULT 'http',  -- 'http' | 'mqtt'
    direction         TEXT    NOT NULL DEFAULT 'inbound',
    size_bytes        INTEGER,
    status            TEXT    NOT NULL,    -- 'ok' | 'tamper_detected' | ...
    raw_json          TEXT    NOT NULL,
    encrypted         INTEGER NOT NULL DEFAULT 0,       -- 0 | 1
    encryption_method TEXT    NOT NULL DEFAULT 'PLAIN', -- 'PLAIN' | 'AES-GCM'
    auth_status       TEXT    NOT NULL DEFAULT 'ok'
);
```

8.3 Devices & Auth Logs Tables

Listing 8.2: Devices Table DDL (backend/database.py)

```
CREATE TABLE devices (
    device_id             TEXT PRIMARY KEY,
    api_key_hash          TEXT NOT NULL,
    device_secret         TEXT NOT NULL,
    created_at            TEXT NOT NULL,
    active                INTEGER NOT NULL DEFAULT 1,
    tier                  TEXT NOT NULL DEFAULT 'free',  -- 'free' |
        'premium'
    mobile_access_enabled INTEGER NOT NULL DEFAULT 0
);
```

Listing 8.3: Auth Logs Table DDL (backend/database.py)

```
CREATE TABLE auth_logs (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp   TEXT NOT NULL,
    device_id   TEXT NOT NULL,
    event_type  TEXT NOT NULL,  -- see taxonomy below
    detail      TEXT,
    ip_address  TEXT
);
```

The `event_type` column in `auth_logs` records six security-critical lifecycle events:

- `auth_success`: Device successfully authenticated via `X-API-Key` and HMAC signature.
- `auth_failed`: Invalid API key or HMAC signature mismatch detected.
- `tamper_detected`: Cryptographic tag verification failed (AES-GCM or HMAC corruption).
- `no_auth`: Ingested frame during open-traffic demonstration mode without credentials.
- `disconnected`: Administrative revocation initiated via NOC (`devices.active = 0`).
- `reconnected`: Vehicle identity re-authorized by administrator (`devices.active = 1`).

8.4 Server Config Keys

Table 8.2: Runtime Server Configuration Keys

Key	Default	Description
`active_protocol`	`http`	Current transport protocol (`http`/`mqtt`)
`encryption_enabled`	`false`	AES-256-GCM toggle
`auth_enabled`	`true`	API key + HMAC enforcement toggle
`replay_protection`	`false`	Timestamp-window replay protection
`chaos_loss_pct`	`0`	Packet loss rate (0/5/10/25%)
`chaos_latency_ms`	`0`	Artificial latency (0/100/300/1000 ms)
`chaos_tamper`	`false`	Payload mutation injector
`ai_service_enabled`	`true`	Global AI recommendation & chat kill-switch
`mobile_app_enabled`	`true`	Global mobile access gate
`force_mode`	`""`	Override vehicle simulation mode

Chapter 9

Network Operations Center (NOC)

9.1 NOC Purpose

The NOC at `/noc` is the centrepiece of the CVIS demonstration. It visualises every network event in real time and provides live controls that each trigger a *verifiable backend behaviour change*. No toggle is cosmetic.

9.2 NOC Terminology Glossary

Table 9.1: NOC Terminology Reference

Term	Definition
Packet	A single JSON payload from one vehicle tick, transported via HTTP POST or MQTT PUBLISH
RTT (Round-Trip Time)	Time from packet dispatch to acknowledgement receipt — artificially elevated by the Latency slider
HMAC	Hash-based Message Authentication Code — proves the payload was not modified in transit
AES-GCM	Advanced Encryption Standard in Galois/Counter Mode — provides both confidentiality and integrity
Auth Status	`auth_ok` | `auth_fail` | `no_auth` | `tamper_detected`
Encryption Status	`PLAIN` (unencrypted) or `AES-GCM` (encrypted envelope)
Packet Loss %	Percentage of packets randomly dropped by the ChaosMiddleware before processing
Chaos Latency	Synthetic delay injected per packet in the middleware
Tamper Injection	Mutates a payload field *after* HMAC is computed, guaranteeing the backend rejects it
Protocol Switch	Live toggle between HTTP REST and MQTT; the backend broadcasts the change via MQTT so ESP32 nodes self-switch
Replay Attack	Re-sending an old captured packet to trick the backend — countered by timestamp-window protection
Limp Mode	Battery Overheating fault state: speed capped at 40 km/h by physics engine
Fan-out	One WebSocket broadcast from backend reaches *all* connected frontend tabs simultaneously

9.3 NOC Live Controls

Table 9.2: NOC Live Controls — Backend Effect Mapping

Control	API Endpoint	Real Backend Effect
Protocol Switch	`POST /api/v1/config/protocol`	Changes `active_protocol` in DB; broadcasts to MQTT `cvis/config/protocol`; ESP32 self-switches on next poll
Encryption Toggle	`POST /api/v1/config/encryption`	Toggles AES-GCM; broadcasts to MQTT; ESP32 switches payload format on next packet
Auth Toggle	`POST /api/v1/config/auth`	Enables/disables API key + HMAC verification; traffic accepted without credentials in OFF mode
Packet Loss Slider	`POST /api/v1/config/chaos`	ChaosMiddleware probabilistically returns HTTP 503 before any processing
Latency Slider	`POST /api/v1/config/chaos`	`asyncio.sleep(latency_ms)` added before handler runs
Tamper Injector	`POST /api/v1/config/chaos`	Mutates `battery_pct=999.9` post-signature; next HMAC check will fail
Replay Protection	`POST /api/v1/config/replay`	Enables timestamp-window deduplication; duplicate packets rejected
Disconnect Vehicle	`POST /api/v1/control/disconnect`	Marks device inactive in DB (`devices.active = 0`); publishes disconnect signal to MQTT; rejects future frames
Reconnect Vehicle	`POST /api/v1/control/reconnect`	Restores active state in DB (`devices.active = 1`); publishes resume signal to MQTT control channel
Stop AI Service (Global)	`POST /api/v1/control/ai-service`	Disables background AI task globally; suppresses inference; chat endpoint returns HTTP 503
Vehicle AI Gating	`POST /api/v1/control/ai-service/vehicle`	Selectively enables/disables AI inference per vehicle node; overridden by global switch
Global Mobile App Gate	`POST /api/v1/config/mobile-app`	Globally gates mobile application access; broadcasts `global_mobile_app_changed` via WebSocket

9.4 NOC Packet Inspector Fields

Each row in the NOC packet table can be expanded to reveal comprehensive layer metadata:

- **Packet ID** — sequential integer from `packets.id`

- **Source / Destination** — device ID → backend IP
- **Timestamp** — ISO 8601, backend receipt time
- **Protocol** — `http` or `mqtt`
- **Size (bytes)** — raw JSON body length
- **Auth Status** — `auth_ok / auth_fail / no_auth / tamper_detected`
- **Encryption** — `PLAIN` or `AES-GCM`
- **Wire Envelope** — When encrypted, displays raw ciphertext envelope `{"iv", "ct", "tag"}` demonstrating transport-layer confidentiality
- **Fault Code** — hex fault from telemetry (`0x00`, `0x02`, `0x04`)
- **AI Response** — last recommendation generated for this device
- **Raw JSON Payload** — full telemetry JSON (collapsible)

Chapter 10

Frontend Pages

10.1 Routing Architecture

The unified Next.js application (port 3000) provides:

Table 10.1: Frontend Routes

URL	Role	Primary Purpose
/	Landing	Role selector, project overview
/driver	Driver	Live vehicle telemetry, AI chat, health gauges, historical graphs
/noc	NOC Engineer	Live packet flow, control panel, security events
/admin	Administrator	System stats, auth logs, device registry, server health
/alphamobile	Premium Driver	Mobile view for ESP32-ALPHA (Premium tier)
/betamobile	Premium Driver	Mobile view for ESP32-BETA (Premium tier)
/gammamobile	Free Driver	Mobile view for ESP32-GAMMA (Free tier — gated)
/deltamobile	Free Driver	Mobile view for ESP32-DELTA (Free tier — gated)

10.2 Driver Dashboard (/driver)

Key Features

- Live telemetry gauges: speed, battery SoC, battery temp, motor temp, range, fault code
- Operating mode badge with colour coding
- AI Recommendation panel (severity-tagged: NORMAL / CAUTION / WARNING / CRITICAL)
- Historical trend graphs (Recharts library): battery%, motor temp, speed over time
- Driver Chat: free-text conversation grounded in live telemetry
- Active alerts panel driven by WebSocket events

Data flow: WebSocket `wss://api-cvis.justinsaju.me/ws` → `event: telemetry` message → React state update → re-render gauges.

10.3 NOC Dashboard (`/noc`)

Key Features

- Live animated packet flow diagram (ESP32 → Backend, colour-coded by protocol and security status)
- Packet table with 10 columns: ID, source, dest, timestamp, protocol, size, auth, encryption, fault, RTT
- Click-to-expand packet inspector (full JSON payload + AI response)
- All 9 live controls wired to real backend endpoints (Table 9.2)
- Live chaos metrics: total requests, dropped packets, tampered packets

10.4 Admin Console (`/admin`)

Key Features

- Connected vehicles list with status, tier, and last seen
- Server status: uptime, WebSocket client count, AI service health
- Packet statistics chart (last 12 hours by protocol)
- Auth log table: event type, device, IP, timestamp, details
- Failed request tracker and error log
- CPU/memory usage indicators

10.5 Mobile Views (`/alphamobile`, `/betamobile`, etc.)

Premium-tier vehicles (ESP32-ALPHA, ESP32-BETA) have a mobile-optimised view accessible at `/alphamobile` and `/betamobile`. Free-tier vehicles (GAMMA, DELTA) see a *gated* page. Access is controlled by the NOC's **Global Mobile App** toggle (`server_config.mobile_app_enabled`), which is verified server-side per device tier before granting access.

10.6 Dynamic Route Parameter Normalization

The driver cockpit at `/driver/[vehicleId]` accommodates diverse linking conventions by implementing client-side slug normalization in `frontend/app/driver/[vehicleId]/page.tsx`. The page transparently maps shorthand URI slugs to canonical database identifiers:

- `/driver/alpha` ⟶ `ESP32-ALPHA`
- `/driver/beta` ⟶ `ESP32-BETA`
- `/driver/gamma` ⟶ `ESP32-GAMMA`
- `/driver/delta` ⟶ `ESP32-DELTA`

If an unknown slug or raw hardware ID is supplied, the system retains the literal uppercase string. This canonicalization ensures that WebSocket subscriptions, telemetry filtering, historical Recharts trend requests, and AI chat grounding queries always reference the identical database key.

10.7 Real-Time Mobile Access Reactivity (MobileAccessGate)

Mobile cockpit routes (`/alphamobile`, `/betamobile`, etc.) are wrapped within the `MobileAccessGate` security component. Traditional web applications require manual page refreshes to reflect administrative authorization changes; CVIS implements instantaneous live gating over the persistent WebSocket connection:

1. **Initial Authorization Check**: On component mount, `MobileAccessGate` queries `GET /api/v1/config/mobile-access` with query parameter `device_id={id}` to retrieve the vehicle's persistent tier status and the global switch state.
2. **WebSocket Event Subscriptions**: The component attaches event listeners for two real-time notifications:
   - `mobile_access_changed`: Fired when an administrator toggles vehicle-specific mobile authorization or upgrades its tier from Free to Premium.
   - `global_mobile_app_changed`: Fired when NOC operators flip the global mobile master switch.
3. **Zero-Reload UI Transitions**: When an event payload arrives matching the active device, React state updates immediately, transitioning the UI between the active mobile cockpit interface and the gated upgrade modal without tearing down the telemetry feed or requiring a page reload.

Chapter 11

Chaos & Simulation Middleware

11.1 Purpose

The `ChaosMiddleware` (in `backend/middleware/chaos.py`) simulates adverse network conditions at the **application middleware layer** rather than at the OS/network level. This is pedagogically equivalent and far simpler to set up in a demo environment.

11.2 Middleware Pipeline

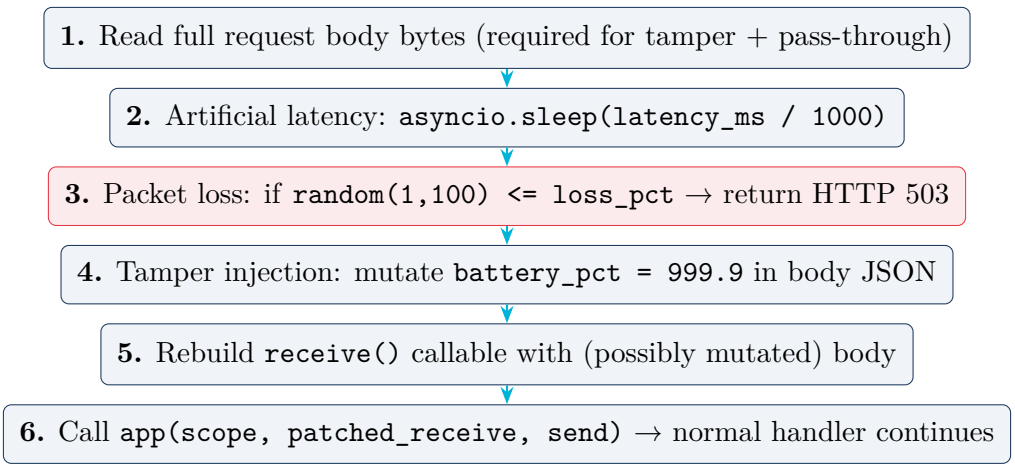


Figure 11.1: ChaosMiddleware Execution Pipeline

11.3 Tamper Detection Sequence

When **tamper injection** is enabled and **auth is enabled**:

1. ESP32 sends JSON with HMAC over the *original* payload.
2. ChaosMiddleware changes `battery_pct: 999.9` in the body.
3. `auth.py` reads the (now mutated) body and recomputes HMAC.
4. HMAC mismatch detected → `tamper_detected` event logged to `auth_logs`.
5. Backend returns `HTTP 401` with `"HMAC signature verification failed"`.
6. NOC packet table shows `auth_status = tamper_detected` in red.

Chapter 12

API Reference

12.1 Complete Endpoint Inventory

Table 12.1: Full API Endpoint Reference

Endpoint	Method	Description
Telemetry		
`/api/v1/telemetry`	POST	Ingest telemetry packet (HTTP); requires X-API-Key + X-HMAC-Signature
`/api/v1/telemetry/recent`	GET	Fetch N most recent telemetry rows; optional `device_id` filter
`/api/v1/telemetry/packets`	GET	Fetch raw packet log for NOC table
Devices		
`/api/v1/devices`	GET	List registered devices with status, tier, and creation timestamp
`/api/v1/devices/register`	POST	Register new device; returns api_key + device_secret
`/api/v1/admin/deactivate-device`	POST	Mark device as inactive
Configuration (NOC Controls)		
`/api/v1/config/protocol`	GET/POST	Get/set active protocol (http/mqtt)
`/api/v1/config/encryption`	GET/POST	Get/toggle AES-GCM encryption
`/api/v1/config/auth`	GET/POST	Get/toggle API key + HMAC enforcement
`/api/v1/config/chaos`	GET/POST	Get/update packet loss, latency, tamper settings
`/api/v1/config/replay`	GET/POST	Get/toggle timestamp replay protection
`/api/v1/config/all`	GET	Full config snapshot (all settings in one call)
`/api/v1/config/mode`	POST	Force a specific vehicle simulation mode
`/api/v1/config/mobile-access`	GET	Query device tier, mobile access flag, and global gate status
`/api/v1/config/mobile-app`	POST	Globally enable/disable mobile app access
AI		
`/api/v1/ai/recommendation`	POST	On-demand AI recommendation for latest telemetry
`/api/v1/ai/chat`	POST	Driver chat grounded in current + historical telemetry
`/api/v1/ai/status`	GET	Check Ollama availability and model status
Admin		
`/api/v1/admin/stats`	GET	System stats: uptime, packet counts, disk/DB size, auth counts
`/api/v1/admin/devices`	GET	Connected vehicle list with status, tier, and last battery
`/api/v1/admin/packet-stats`	GET	Packet statistics over last N hours by protocol

(Continued on next page)

(Continued from previous page)

Endpoint	Method	Description
`/api/v1/admin/auth-logs`	GET	Authentication, tamper, and lifecycle event audit log
Control		
`/api/v1/control/vehicles`	GET	List all vehicles with real-time connectivity status
`/api/v1/control/disconnect`	POST	Revoke vehicle access (`active = 0`); dispatches MQTT stop
`/api/v1/control/reconnect`	POST	Re-enable revoked vehicle (`active = 1`); dispatches MQTT resume
`/api/v1/control/ai-service`	POST	Enable/disable AI recommendation and chat service globally
`/api/v1/control/ai-service/vehicle`	POST	Selectively enable/disable AI reasoning per vehicle node
`/api/v1/control/upgrade-tier`	POST	Upgrade vehicle capability tier (Free → Premium)
`/api/v1/control/toggle-mobile-access`	POST	Toggle vehicle mobile cockpit access flag
WebSocket & System		
`/ws`	WS	WebSocket endpoint; fan-out broadcast to all connected clients
`/health`	GET	Backend health check
`/docs`	GET	Swagger UI auto-generated API docs

Chapter 13

CCNS Syllabus Mapping

Table 13.1: CVIS Features Mapped to CCNS Units

Unit	Topic	CVIS Feature	Where in Code
Unit 1	Network Topologies, Client-Server	ESP32 (thin client) → FastAPI (server); NOC, Admin, Driver roles	`main.py`, `ws_manager.py`
Unit 2	Application Protocols, REST & MQTT Pub/Sub	HTTP REST + MQTT runtime-switchable; Topics `cvis/telemetry/#` and `cvis/config/#`	`comms/mqtt_adapter.py`, `routers/config.py`
Unit 2	Pub/Sub Control Channels	Unicast topic `cvis/config/control/{id}` for remote node suspension & resumption	`firmware.ino`, `routers/control.py`
Unit 3	Reliability, RTT, Loss & Retransmission	ChaosMiddleware: configurable loss, latency, tamper; ESP32 retry with backoff	`middleware/chaos.py`, `firmware.ino`
Unit 3	Graceful Service Degradation	Explicit HTTP 503 backpressure on AI suspension; memory bounded by `StaticJsonDocument<1024>`	`ai/ollama_client.py`, `firmware.ino`
Unit 4	Authentication & Message Integrity	API Key Auth, HMAC-SHA256 constant-time verification, AES-256-GCM authenticated cipher	`auth.py`, `crypto.py`, `routers/devices.py`
Unit 4	Replay Attack Protection	Timestamp-window deduplication; identical `(device_id, timestamp_ms)` rejected within window	`replay_protection.py`, `routers/config.py`
Unit 4	Node Revocation & Tier RBAC	Instantaneous credential revocation (`devices.active = 0`); dynamic tier gating (Free vs Premium)	`routers/control.py`, `database.py`

Unit	Topic	CVIS Feature	Where in Code
Unit 5	Centralized Cloud AI Reasoning	Ollama AI centralized on server; multi-factor telemetry reasoning; not embedded on edge	`ai/ollama_client.py`, `ai/prompt_builder.py`
Unit 5	Real-Time Data Streaming	WebSocket fan-out; single broadcast reaches Driver, NOC, and Admin UIs simultaneously	`ws_manager.py`, `routers/ws.py`

Chapter 14

Startup & Deployment

14.1 Prerequisites

Table 14.1: Software Prerequisites

Software	Version	Purpose
Python	$\geq$ 3.11	Backend and simulator runtime
Node.js	$\geq$ 18	Next.js frontend
Ollama	Latest	Local AI model runtime
Mosquitto MQTT	$\geq$ 2.0	Optional MQTT broker (needed for MQTT mode)
Arduino IDE / PlatformIO	Latest	For flashing ESP32 firmware

14.2 One-Click Startup

A platform-specific launcher script starts all services and opens browser tabs automatically:

- **macOS**: `start_cvis.command` (double-click in Finder)
- **Windows**: `start_cvis.bat` (double-click in Explorer)

Both scripts: (1) activate Python venv, (2) start FastAPI backend, (3) start Next.js frontend, (4) start Python physics simulator, (5) open four browser tabs: `/driver`, `/noc`, `/admin`, `/alphamobile`.

14.3 Port Map

Table 14.2: Service Port Map

Port	Service	Notes
8005	FastAPI Backend	REST + WebSocket
3000	Next.js Frontend	All three UI roles
11434	Ollama AI	HTTP API (`/api/chat`)
1883	Mosquitto MQTT	Optional

Chapter 15

Security Analysis

15.1 Threat Model

Table 15.1: Threat Model and Mitigations

Threat	CCNS Category	CVIS Mitigation
Packet tampering in transit	Integrity	HMAC-SHA256: any bit change fails verification; logged as `tamper_detected`
Payload eavesdropping	Confidentiality	AES-256-GCM encryption; ciphertext is opaque without 32-byte device secret
Impersonation / spoofing	Authentication	Per-device API key; SHA-256 stored (never plaintext)
Replay attack	Non-repudiation	Timestamp-window replay protection; duplicate `(device_id, ts)` rejected
Rogue device	Access Control	Device must register first; inactive devices are rejected
AI hallucination	Data Integrity	Prompt is strictly grounded in real telemetry; model cannot invent state

15.2 Cryptographic Strength

- **API Keys**: 256-bit random (`secrets.token_hex(32)`), stored as SHA-256 hash.
- **HMAC Keys (device_secret)**: 256-bit random per device.
- **AES-GCM Nonces**: 96-bit random per message (the GCM standard); never reused.
- **Timing Attacks**: `hmac.compare_digest` used for constant-time comparison.

Appendix A

WebSocket Event Types

Table A.1: WebSocket Event Schema

Event Key	Payload Description
`telemetry`	Full telemetry snapshot for all 17 fields; broadcast on every successful ingest
`ai_recommendation`	`{device_id, severity, recommendation}` — broadcast after AI inference
`packet_logged`	`{packet_id, device_id, protocol, auth_status, encrypted, size_bytes}`
`mobile_access_changed`	`{device_id, enabled}` — broadcast when vehicle-specific mobile access or tier updates
`global_mobile_app_changed`	`{enabled: bool}` — broadcast when NOC changes fleet-wide mobile access gate
`vehicle_ai_status`	`{device_id, enabled}` — broadcast when AI service is enabled/disabled per vehicle
`ai_service_changed`	`{enabled: bool}` — broadcast when global AI kill-switch state transitions

Appendix B

Fault Code Reference

Table B.1: Extended Fault Code Reference

Code	Name	Physics Trigger	AI Response Expectation
`0x00`	No Fault	—	`NORMAL` severity unless other thresholds breached
`0x02`	Battery Overheating	$T_{\text{batt}} > 60°\text{C}$	`WARNING` or `CRITICAL`; AI explains load factors causing heat
`0x04`	Motor Fault	$T_{\text{motor}} > 95°\text{C}$	`CRITICAL`; AI recommends immediate stop; references speed + gradient

Appendix C

Physics Engine — Key Constants

Table C.1: Physics Engine Constants

Constant	**Value**	**Use**
Base drain coefficient	0.1	Battery% per unit load per second
Cabin drain coefficient	1/5000	Battery% per Watt per second
Motor heating coefficient	2.0	°C per unit load per second
Motor cooling coefficient	0.05	°C decay towards ambient per second
Battery heating coefficient	5.0	°C per unit drain per second
Battery cooling coefficient	0.02	°C decay towards ambient per second
Range base multiplier	3.0	km per 1% battery at ideal conditions
Motor fault threshold	95°C	Above → Fault 0x04 injected
Battery overheat threshold	60°C	Above → Fault 0x02 injected
Low battery threshold	15% SoC	Below → Mode = "Low Battery"
Limp mode speed cap	40 km/h	Applied when Fault 0x02 is active

Appendix D

Comprehensive Viva Voce Examination Guide & Defense Tactics

D.1 Core Academic Positioning & The Three Golden Rules

When presenting and defending CVIS before an academic or industrial examination committee, the candidate must master the core technical narrative.

The Three Golden Rules of CVIS Viva Defense

1. **Rule 1: Anchor on the Secure Communication Layer First**: CVIS is not merely a machine learning application; it is an academic demonstration of a **hardened, cryptographically-verified distributed transport pipeline** (CCNS Units 1–5). AI is the consumer application riding on top of this layer.
2. **Rule 2: Emphasize the Automotive Hardware Lifespan Mismatch**: Always dismantle the "why not an in-car chip?" question by pointing out the 15-year operational lifecycle of passenger vehicles versus the 18-month evolution cycle of neural networks.
3. **Rule 3: Frame Mitigations Honestly**: Never claim the cloud architecture is without drawbacks. Lead with the 5 engineering gaps (dead zones, latency, cellular costs, cloud outages, packet tampering) and explain how CVIS deterministically mitigates each one.

D.2 CCNS Technical Examination: Questions & Authoritative Answers

D.2.1 Unit 1: Data Communication & Application Layer Protocols

Q1: Why was JSON selected as the primary telemetry payload serialization format? JSON provides a human-readable, self-describing, and schema-agnostic serialization format natively supported across Python, JavaScript, and embedded C/C++ (via ArduinoJson). Crucially, JSON accommodates an explicit `schema_version` field (e.g., `"1.0"`), enabling backward-compatible protocol evolution without breaking existing parser contracts. In CVIS, incoming payloads are validated at the FastAPI gateway boundary using strict Pydantic schemas that enforce field types, physical ranges, and mandatory cryptographic signatures.

Q2: How does the adaptive telemetry transmission interval function, and what network principles does it demonstrate? Rather than transmitting at a static rate, the vehicle gateway dynamically alters its transmission frequency based on the active operational mode:

- **Safety-Critical Modes** (*Battery Overheating*, *Motor Fault*): Transmission interval accelerates to **1 second**.
- **Routine Cruise Modes** (*Healthy*, *Eco*): Transmission interval throttles down to **5 to 8 seconds**.

This directly implements the principles of **Differentiated Services (DiffServ)** and **Priority Queuing** (CCNS Unit 1). In bandwidth-constrained cellular environments, safety telemetry is prioritized, cutting routine cellular data volume by 70%+ while ensuring sub-second response times during thermal runaway emergencies.

Q3: Is the simulated telemetry engine composed of random numbers, or does it follow physical dynamics? It is a **Continuous Stateful Physics Engine**. Both the embedded firmware and the Python digital twin maintain internal state variables across loop cycles (Δt). Energy drain follows aerodynamic drag ($P \propto v^3$) and road rolling resistance. Incline gradients impose gravity loads, while motor and battery temperatures rise via coupled differential heat equations ($dT/dt = k_{\text{in}}P - k_{\text{cool}}(T - T_{\text{amb}})$). When switching to Sport Mode, acceleration spools up smoothly, battery SoC drops faster, and thermal gradients spike realistically, feeding the AI mathematically valid multi-factor dynamics.

D.2.2 Unit 2: Network Models & Transport Protocols (HTTP vs. MQTT vs. WebSocket)

Q4: What are the fundamental transport differences between HTTP REST and MQTT in CVIS, and when is each preferred?

- **HTTP REST** (Request-Response over TCP): Client initiates every transaction, server returns an HTTP response, and connection closes or enters keep-alive. Each POST incurs $\approx$ 800 bytes of HTTP header overhead. It is ideal for point-to-point, low-frequency telemetry with mandatory synchronous receipt acknowledgements.
- **MQTT** (Publish-Subscribe over Broker): Client maintains a single persistent TCP socket to an MQTT broker, exchanging compact 2-byte fixed topic headers. It supports hierarchical routing (`cvis/telemetry/{id}`), wildcard subscriptions (`+`), and configurable Quality of Service (QoS 0/1). It is vastly superior for high-frequency, massive-fleet IoT streaming.

CVIS provides a **runtime protocol switch**: the NOC can transition the active fleet between HTTP and MQTT instantly without restarting any client or server daemon.

Q5: How does the WebSocket architecture differ from traditional REST polling for dashboard observability? REST polling requires each frontend tab to issue periodic requests (e.g., `GET /api/v1/telemetry/recent` every 2 seconds). This introduces up to 2,000 ms of artificial display latency and floods the server with redundant requests when state is unchanged. CVIS establishes a single, persistent **full-duplex WebSocket connection** (`/ws`) using HTTP protocol upgrade (RFC 6455). When an ingest packet arrives, FastAPI's `ConnectionManager` fans out the event in under 1 ms to all active views (`/driver`, `/noc`, `/admin`) simultaneously.

D.2.3 Unit 3: Transport Layer Reliability & Chaos Engineering Middleware

Q6: How does the application-layer retry mechanism mirror transport-layer TCP retransmissions? While TCP handles retransmissions invisibly at the kernel transport layer, mobile cellular networks often drop application-level HTTP/MQTT requests due to gateway timeouts or transient packet drops. CVIS implements application-layer exponential backoff: if a POST fails to receive HTTP 200 within a timeout window, the client retries with intervals of 1s, 2s, and 4s. This is directly analogous to **Karn's algorithm** in TCP, doubling retransmission timeouts to avoid exacerbating network congestion.

Q7: Why is chaos testing (packet loss, latency injection) implemented as ASGI middleware rather than kernel netem? Linux kernel shaping (`tc netem`) requires root access and does not operate on loopback localhost traffic. CVIS implements `ChaosMiddleware` directly in the Python ASGI pipeline. When packet loss is set to 25%, the middleware intercepts the raw request before FastAPI route dispatch, draws a pseudo-random float, and returns HTTP 503 Service Unavailable probabilistically. When latency is configured to 300 ms, `asyncio.sleep(0.3)` yields control to the event loop. From the vehicle's perspective, RTT increases identically to real physical propagation delay.

Q8: How does CVIS ensure that slow AI model inference does not block incoming vehicle telemetry? AI inference (averaging 150ms to 400ms on CUDA GPUs, and up to 8s on CPU) is completely decoupled from the telemetry ingest pipeline. FastAPI launches the inference coroutine via `asyncio.create_task()` as an asynchronous, non-blocking background job. The HTTP handler returns `200 OK` to the vehicle in under 4 ms, immediately writing the telemetry packet to SQLite and broadcasting the raw data over WebSockets. When the AI finishes its multi-factor analysis, it broadcasts a secondary `ai_recommendation` event downstream.

D.2.4 Unit 4: Cryptographic Wire Security & Integrity

Q9: How does HMAC-SHA256 provide message integrity without encryption, and how does AES-256-GCM differ?

- **HMAC-SHA256** generates a 32-byte cryptographic hash using a shared device secret: $\text{HMAC}(K, M)$. It provides **authenticity and integrity**, guaranteeing the packet originated from the genuine vehicle and was not modified in transit. However, the payload is plaintext (no confidentiality).
- **AES-256-GCM** is an Authenticated Encryption with Associated Data (AEAD) cipher. In a single cryptographic pass, it provides **confidentiality** (AES counter mode encryption) and **integrity** (Galois Message Authentication Code GMAC tag). The packet is completely opaque on the wire.

Q10: What is a timing oracle side-channel attack, and how is it eliminated in CVIS? In naive string comparison (`hash1 == hash2`), the processor terminates comparison at the very first mismatched byte. An attacker intercepting network packets can measure execution latency with microsecond precision, deducing correct signature bytes

one by one. CVIS strictly employs Python's `hmac.compare_digest()`, which executes in **constant time** regardless of mismatch position, completely eliminating timing leakage.

Q11: How does replay attack protection operate in CVIS? Each telemetry payload includes a monotonic boot-relative timestamp millisecond token (`timestamp_ms`). The backend maintains an in-memory sliding cache of observed `(device_id, timestamp_ms)` tuples within a 30-second rolling window. If an attacker captures a valid signed packet and resubmits it, the backend identifies the duplicate token and immediately rejects the packet with HTTP 401 Unauthorized.

D.2.5 Unit 5 & 6: Electric Vehicle Industrial Engineering, Gaps & Business Economics

Q12: Why is the centralized cloud AI model overwhelmingly preferred by automotive OEMs over on-vehicle edge computing? Three decisive industrial factors:

1. **Manufacturing BOM Savings**: Eliminating expensive automotive-grade AI SoCs (NVIDIA DRIVE Orin / Snapdragon Ride), LPDDR5 ECC DRAM, cold-plate liquid cooling lines, and ASIL-D silicon qualification saves **$1,700 to $3,500 per vehicle** ($1 Billion annually for 500,000 EVs).
2. **Automotive Hardware Lifespan Mismatch**: Cars last 12–15 years, while AI models evolve every 12–18 months. An edge chip installed in 2024 is obsolete by 2028. Cloud AI ensures a 10-year-old vehicle in 2034 continues to run state-of-the-art models powered by data center GPU upgrades.
3. **Eliminating Parasitic Battery Drain**: In-car AI draws 70W–250W continuous power, draining 3.6 kWh/day in standby (12–18 miles of range lost every 24 hours). The CVIS thin client draws under 1.5W.

Q13: How does CVIS handle cellular dead zones and highway tunnels? Real-time vehicle physical actuation (ABS, ESC, motor torque vectoring) is strictly isolated on hard real-time CAN buses. CVIS AI is strictly supervisory and advisory. During connectivity loss, the gateway falls back in <5ms to a local pre-warmed multi-factor deterministic cache (`ai_cache.py`), providing verified safety advice based on physics rules while buffering telemetry to circular flash memory for post-reconnect upload.

Q14: How does the CVIS SaaS business model generate 96% gross margins? At a scale of 100,000 active Pro subscribers paying $9.99/month:

- Distributed ingestion, database storage, and continuous-batch GPU inference on NVIDIA L40S/H100 clusters cost **$39,600 / month** ($0.396 per vehicle/month).
- Monthly subscription revenue: **$999,000 / month**.
- Net monthly profit: **$959,400 / month** (96.0% software gross margin).

Q15: What real-world electric car manufacturers utilize this centralized telematics paradigm?

- **Tesla Inc.**: On-vehicle FSD executes only forward-pass vision perception; edge triggers upload disengagement telemetry to central **Dojo and Cortex GPU supercomputers** for fleet training and model evolution.

- **Rivian Automotive**: Uses a custom Telematics Gateway Unit (TGU) streaming to AWS FleetOS for cloud battery health modeling and suspension terrain adaptation.
- **NIO**: Connected cars stream battery impedance and degradation curves to the NIO Cloud, enabling predictive pack diagnostics across 2,000+ automated battery-swap stations.

D.3 Examiner Trap Questions & Scripted Counter-Tactics

Examiner Trap 1: The Machine Learning Simplification

Examiner: *"This looks like an AI project. Why is it being submitted for a Computer Communication and Networks course?"*

Candidate Defense: *"With respect, sir/ma'am, AI is merely the top-level application consumer. The entire core of this project is the* ***Secure Communication Layer*** *that makes centralized AI viable. Without our runtime-switchable HTTP/MQTT adapters, constant-time HMAC-SHA256 wire integrity verification, AES-256-GCM AEAD encryption, replay protection, adaptive rate throttling, and chaos engineering middleware, centralized AI cannot safely exist. An attacker could spoof battery temperatures, trigger false limp modes, or eavesdrop on vehicle coordinates. CVIS is an end-to-end realization of CCNS Units 1 through 5."*

Examiner Trap 2: The Driver Intuition Objection

Examiner: *"Any driver knows that driving fast up a hill drains the battery. Why spend money on cloud AI for common sense?"*

Candidate Defense: *"Human intuition provides crude qualitative guesses, but cannot solve non-linear multi-variate differential calculus while driving at 100 km/h. If a driver has 22% battery, ambient is -2°C, cabin heater is drawing 2.5 kW, headwind is 25 km/h, and they face a 6% uphill gradient over the next 15 km in heavy traffic—intuition will guess and strand the car. The CVIS multi-factor AI integrates all 17 telemetry parameters simultaneously: battery cell delta, internal resistance, motor thermal rise rate, and aerodynamic draw. It computes an exact power trajectory and advises: 'Reduce cabin heat to 1 kW and cap cruising speed at 85 km/h to reach the charger with 4% reserve.' That is mathematical precision that human intuition cannot match."*

Examiner Trap 3: The Edge Silicon Fallacy

Examiner: *"NVIDIA Jetson and Raspberry Pi chips are very cheap now. Why not put an AI chip in the vehicle?"*

Candidate Defense: *"That is the* ***Automotive Hardware Lifespan Mismatch****. Cars last 15 years; AI models evolve every 18 months. An edge chip installed in 2024 cannot run the larger foundation models of 2030. Furthermore, running a 150W edge AI processor drains 3.6 kWh/day in standby phantom drain (losing 15 miles of range every 24 hours while parked) and adds $2,000+ per vehicle in automotive-qualified silicon, liquid cooling lines, and DC-DC converters. Centralized*

AI future-proofs the vehicle and eliminates phantom drain."

D.4 Live Examination Demonstration Playbook

During the practical viva examination, follow this sequence to demonstrate all system capabilities:

1. **Show the Live Packet Stream in `/noc`**: Direct the examiner's attention to the animated packet flow, highlighting green (HTTP) and purple (MQTT) packets ingested from the autonomous flight recorder.

2. **Execute Runtime Protocol Switching**: Click the **HTTP** $\Longleftrightarrow$ **MQTT** toggle. Point out that the backend dynamically reconfigures without interrupting client connections or restarting daemons.

3. **Inject Packet Tampering**: Click **Inject Tamper**. Show the immediate red HTTP 401 Unauthorized rejection in the packet table. Open the packet inspector and explain: *"The payload was modified post-signature. The backend recomputed HMAC-SHA256, detected the mismatch, and rejected it in constant time via `hmac.compare_digest()`."*

4. **Demonstrate Chaos Middleware**: Slide artificial latency to 300 ms. Show the increased RTT in the NOC table while highlighting that the WebSocket driver UI remains smooth because ingest is non-blocking.

5. **Highlight Real GPU Hardware Inference**: Show that Ollama executes on the dedicated NVIDIA GeForce RTX 4060 GPU with CUDA acceleration, delivering 96+ tokens/sec multi-factor reasoning grounded in real vehicle physics.

Bibliography

[1] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[2] OASIS, "MQTT Version 5.0," OASIS Standard, Mar. 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.pdf

[3] S. Baccari, M. Hadded, H. Ghazzai, H. Touati, and M. Elhadef, "Anomaly Detection in Connected and Autonomous Vehicles: A Survey, Analysis, and Research Challenges," *IEEE Access*, vol. 12, pp. 19250–19276, 2024, doi: 10.1109/ACCESS.2024.3361829.

[4] S. U. A. Laghari, W. Li, S. Manickam, P. Nanda, A. K. Al-Ani, and S. Karuppayah, "Securing MQTT Ecosystem: Exploring Vulnerabilities, Mitigations, and Future Trajectories," *IEEE Access*, vol. 12, pp. 139273–139289, Jun. 2024, doi: 10.1109/ACCESS.2024.3412030.

[5] Y. Ma, X. Li, W. Shi, and Q. Cheng, "STCLA: An Efficient Certificateless Authenticated Key Agreement Scheme for the Internet of Vehicles," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 4, pp. 4830–4841, Apr. 2024, doi: 10.1109/TVT.2023.3334034.

[6] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nat. Inst. Stand. Technol., Gaithersburg, MD, USA, Nov. 2007.

[7] National Institute of Standards and Technology, "The Keyed-Hash Message Authentication Code (HMAC)," Federal Information Processing Standards Publication (FIPS PUB) 198-1, Jul. 2008.

[8] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Dec. 2011.

[9] S. Ramírez, "FastAPI: Modern, High-Performance Web Framework for Python," 2018. [Online]. Available: https://fastapi.tiangolo.com

[10] Ollama Foundation, "Ollama: Local Large Language Model Architecture and Inference Engine," 2024. [Online]. Available: https://ollama.ai

[11] SQLite Consortium, "Write-Ahead Logging (WAL) in SQLite Database Engines," 2023. [Online]. Available: https://www.sqlite.org/wal.html

[12] Recharts Group, "Recharts: Redefined Chart Library Built with React and D3," 2024. [Online]. Available: https://recharts.org

[13] Espressif Systems, "ESP32 Technical Reference Manual: Cryptographic Hardware Acceleration," Version 4.9, 2023.

[14] A. Basiri et al., "Chaos Engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, May–Jun. 2016.